

المحاضرة الخامسة خوارزميات

B-tree (Bayer tree)

هي عبارة عن tree مخزنة على Hard Disk مثلاً 20 مقارنة
على هارد مكلفة بالإضافة إلى الروحة والرجعة (حلب المعلومات)
والقراءة وخصوصاً إذا كان Hard عبارة عن قرص قديم غير
السرعة راج وقف عليه 20 عليه هي المفروض بسيطة وغير
مكلفة بعد على هارد لا فإحسب الفترة من B-tree من
تقليل عدد ~~البيانات~~ الزهابات إلى Hard Disk قدر لا وكان
من خلال التوجيه بلك كامل من البيانات
والقيام بعملية جلب وإدراج وحذف بتعقيدات $\log$
حتى عندما تكون البيانات كبيرة جداً مثال: (قواعد المعطيات
في أنظمة الملفات - الملفات - الفهارس)

الفترة فيها التوزيع أكثر من عقدة سواء كان واحد
سواء صفحة (page) + لم تعد Binary tree يعني
للأب أكثر من ابنين فالصفحة هي مصفوفة عقدة
ومؤشر على صفحة أخرى وهي Balanced tree
لازم تحقق أكثر من شرطين:

* ال tree من مرتبة t لازم تحقق:

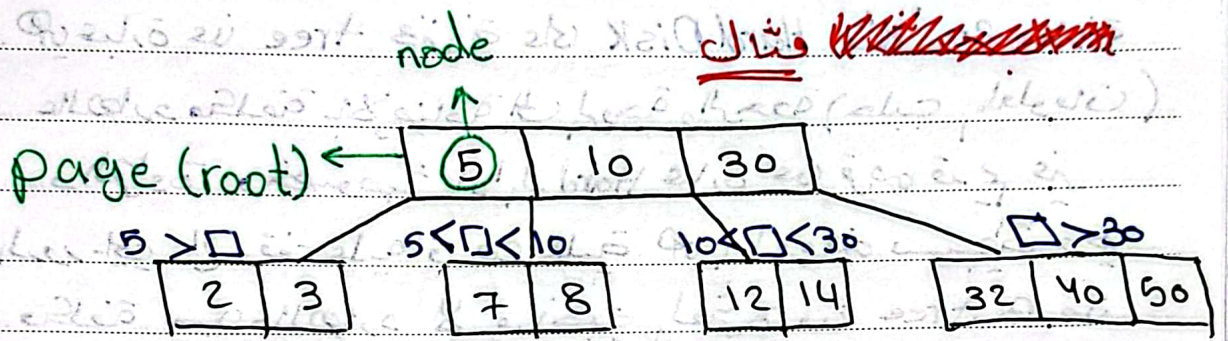
[1] يوجد حد أدنى للعقد في الصفحة الواحدة وهو t

باستثناء الجذر الحد الأدنى تبعو 1

[2] يوجد حد أقصى للعقد في الصفحة الواحدة وهو $2t$

بما فيها صفحة الجذر

- ١٣] العقد داخل الصفحات مرتبة
 ١٤] جميع الأوراق تقع على نفس المستوى
 ١٥] الصفحة لـ m عقدة و $m+1$ ابن أو لابن



« البحث »
 أول شيء يدور على الصفحة وسنلاحظ في الصفحة بدورها
 إما خطأ أو نجاء، القيم مرتبة من أعلى Binary Search



بدي دور على 12 مثلاً، بل إن من صفحة root
 إذا ما لفتت الصفحة فيها بدور على صفحة
 البناء المناسبة أي كما لازم انتقل عليها
 سبة إلى root فالصفحة 12 بعد من الصفحة إلى
 لازم انتقل عليها هي الصفحة بين 10 و 30
 بدور داخل خطأ أو Binary Search

سُطر التوقف عند البحث هو أول لورشة ومالصة
الحضر أي عم دور عليه ف بوقف

البقيّة للبحث

كافة البحث هي h x كافة البحث داخل الصفحة
الارتفاع

استنتاجه لفرة
لي بعد هو

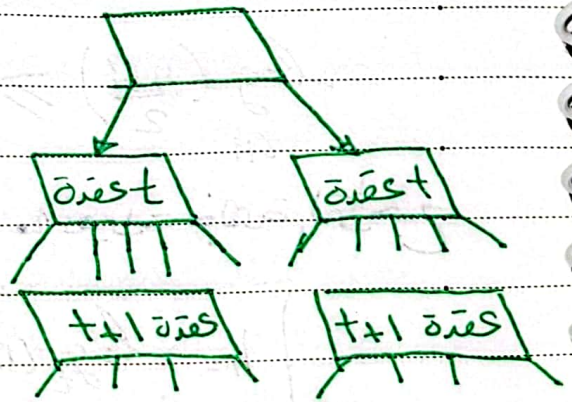
$$h = \log_t(n)$$

$$O(2t \cdot \log_t(n)) = O(\frac{n}{t} \cdot \log_t(n))$$

$$O(\log_2(t) \cdot \log_t(n)) = O(\log_t(n))$$

استنتاج فيّة h بالنسبة لـ n و t عدد لفة في الصفحة

مستوى	عدد الصفحات	عدد لفة
0	1	1
1	2	$2t$
2	$2(t+1)$	$2t(t+1)$
3	$2(t+1)^2$	$2t(t+1)^2$
		$\vdots$



مجموع لفة n

ملاحظة : عم في الجدول يلاحظ بالحد الأدنى لعدد الصفحات

$n \geq 1 + 2t + 2t(t+1) + 2t(t+1)^2$
 لا نرى الحالة، لا سود بيطوع
 بيادي ويمكن تكون مع عدم لعقد أكثر بنفس

$$n \geq 1 + 2t \sum_{i=1}^h (t+1)^{i-1}$$

$$n \geq 1 + 2t \frac{(t+1)^h - 1}{(t+1) - 1}$$

$$n \geq 2(t+1)^h - 1$$

$$\frac{n+1}{2} \geq (t+1)^h$$

$$\log_{t+1} \left(\frac{n+1}{2} \right) \geq h$$

أخذ $\log$
 للطرفين

ال $\log$ تابع متزايد لهلي ما تعديت جهة التراجع

$$h = \log_t(n)$$

ال $\log$ متزايد حالة الارتفاع ليكبر كثير يعني
 ال tree شبه فاحية لهلي الارتفاع

١١.٣ كيف نقيس احسب لو غدا يتم للأساس لي لي أنا
نحتاجو عالة احاسبة

$$\log_t(n) = \frac{\log_{2/10}(n)}{\log_{2/10}(t)}$$

«الإضافة»

أول شيء لازم اعل خب مكان حدد صفحة الإضافة
المناسبة والإضافة صغراً في الأوراق

← إذا كانت م م معينة فيها مكان ل يعني أقل من $2t$

نصف المكان المناسب بالترتيب وخالصين

← إذا معينة مافي مكان يعني أكثر من $2t$ هارت هون

نقسم ~~وطلع~~ الصفحة لنصفين وطلع عندهر المنتصف

على صفحة الأب حتى لو كانت $root$ بإنشئ روت جديد

ليس نقيم في لما قسم الصفحة لفين لي بالنفس وطلعو على الأب !!

حتى اتفقنا الو عدد الأبناء أكبر من عدد العقد في صفحة

الأب بمقدار 1 أنا لما قسمت الصفحة نفين زودت عدد

الأبناء فصاروا أكثر من عقد الأب ب 2 بهل حالة

اختل الشرط لتعديل الشرط بطلع عندهر المنتصف

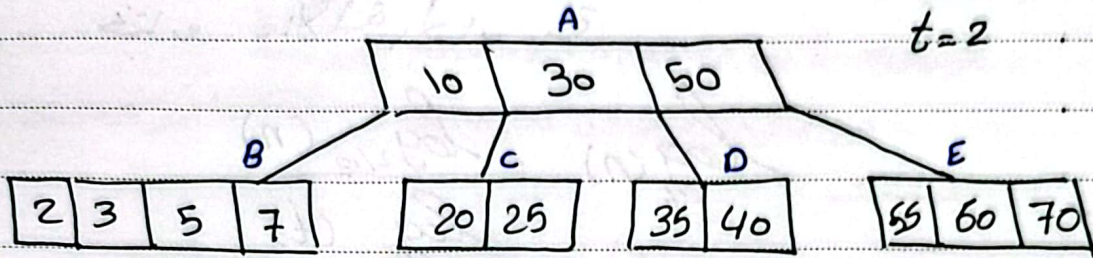
لعد صفحة الأب ف يرجع للحالة الصح الو الأبناء

أكبر من عقد الأب بي بمقدار 1

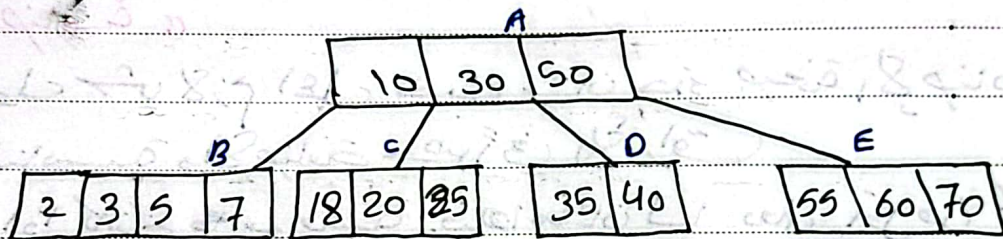
يعني زودت الأبناء ب 1 فزيد الأب

مثال

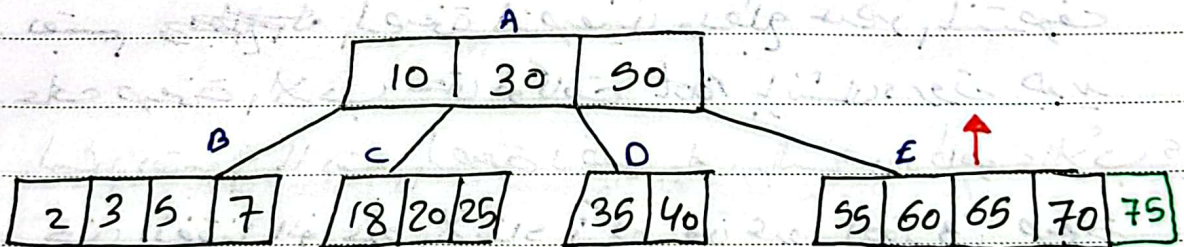
$t=2$



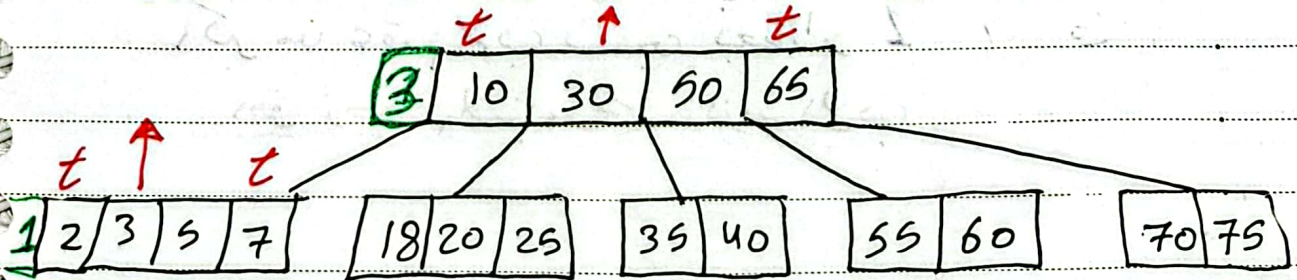
إضافة 18 جديد صفحة، الإضافة هي الصفحة C
الإضافة في الترتيب المناسب للعقدة لأن الصفحة أقل من $2t$



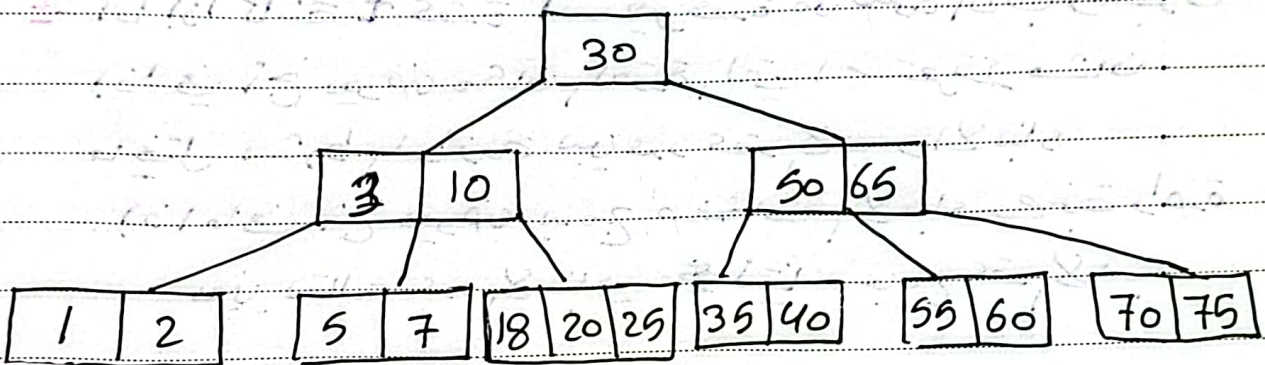
إضافة 65 جديد صفحة، الإضافة هي الصفحة E والصفحة فيها
مكان أقل من $2t$ بحيث يمكن المناسب بالترتيب



إضافة 75 جديد صفحة، الإضافة هي الصفحة E بـ t
صارت أكثر من $2t$ معينة بقسم الصفحة E لصفحتين
كل وحدة t وعند المنتصف بطلع على صفحة الجاب
إذا مومعينة (هون مومعينة في طلع العنبر غير)



إضافة 1 جديد صفحة لبحث B، ولكننا ~~هنا~~ فصلت
 نعتم الصفحة لصغين كل وحدة t وطلع عنصر المنتصف
 إلى صفحة الـ B إذا كانت أقل من $2t$ (هون مو أقل
 فبقسم الصفحة لصغين كمان وطلع عنصر المنتصف لي
 الـ root الجديد للشجرة)



التعقيد
 هي فقط تكلفة الإصلاح h و تكلفة لاني مكان لا إضافة
 h لأنو حتى بالأوراق عم انزل h أمار لا زاجة بجائو الـ t
 هو ثابت كلشي بيعلق فيه $O(1)$ + تكلفة إذا رفعت عنصر

على الـ root h

$$\rightarrow O(h + h + h) + O(1)$$

$$\rightarrow O(h)$$

$$\rightarrow O(\log_t(n))$$



Keep going

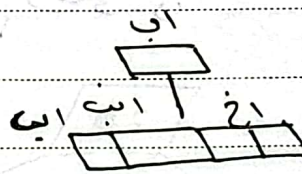
الحذف

يوجد للحذف حالتين إما حذف من ورقة أو حذف من عقدة داخلية

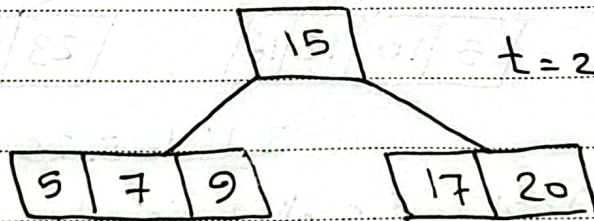
الحذف من ورقة

إذا كانت كوي أكثر من t عقدة ~~حذف~~ حذف عادي بدون أي مشاكل

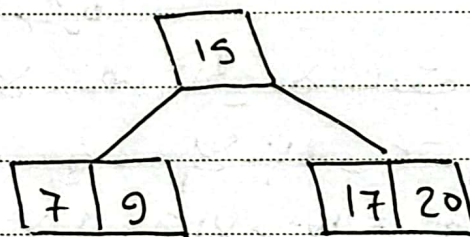
إذا كانت t عقدة ~~حذف~~ أول شيء نستعير من t أب وسنوف عني حالتين إما إذا عني أخ يعوض مكان العقدة t أي أخذت من t أب برفعهما إذا عني بدعج الصفحتين يعني إذا في t أخ

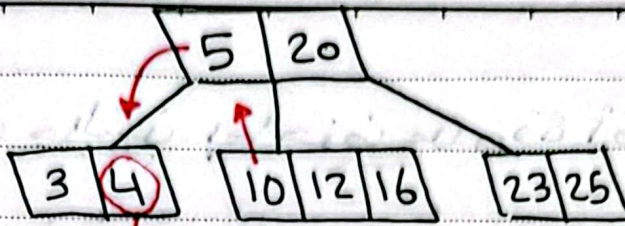


أفقلة

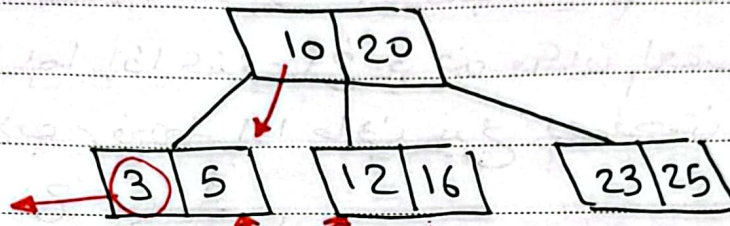


حذف 5 من ورقة 5 أكثر من t حذف دون أي مشاكل

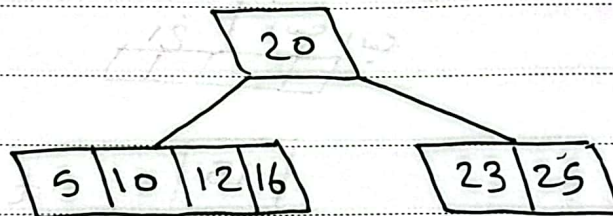




حذف 4 في صفحة ورقة لكن الصفحة تحتوي + عقدة
 1- نستعيد من الـ لاب 2- سنوف إذا في أخ بيخودنا أولا
 هون في أخ يعوضنا منطوحو مكان لاب



حذف 3 في صفحة ورقة لكن الصفحة تحتوي + عقدة
 1- نستعيد من الـ لاب 2- سنوف إذا في أخ بيخودنا
 هون ما في ندمج الصفحتين



2- الحذف من عقدة داخلية

العضو يلي بيدي احذفو في الو عنصر تالي (أكبر منو)
 أو سابق (أصغر منو) قابل
 الحذف العنصر مباشرة

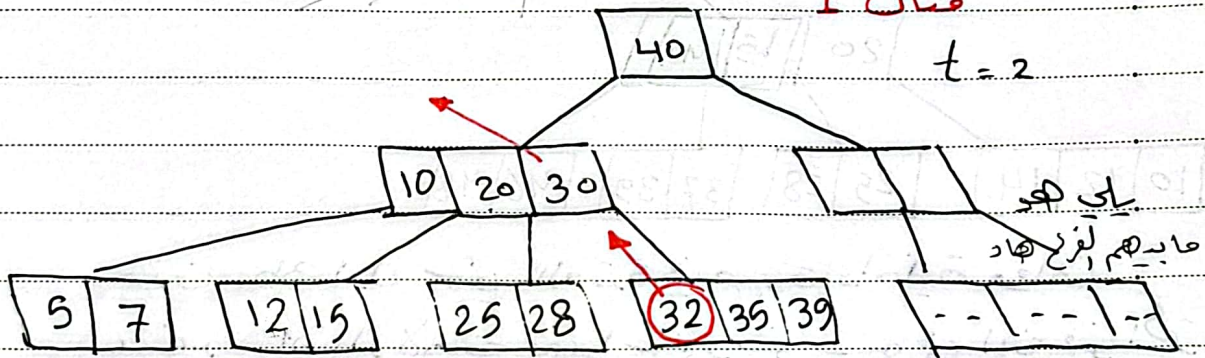
العضو ماخذو لا عنصر تالي ولا سابق ف بس بدمج
 الابداء حتى وازن بين عدد الابداء و عقدة الـ لاب
1, 2 ورقة احذف الأولوية للاستعارة بعدين إذا ما في
 بدمج الابداء

لـ صفحة العقدة أي بي احذف منها عندها ابن فين
 أكثر من t عقدة حسن استيعر منو ؟ بس بشرط
 حافظا على الترتيب + هو هنوي يكون الابن سابق
 أو تالي للعقدة المهم احدا بناء الصفحة
 لـ صفحة العقدة أي بي احذف منو ابناؤها ~~والابن~~
 عندهن t عقدة عا حبن استيعر منهن فز بد مع الابن والاخ
 والابن وكذا

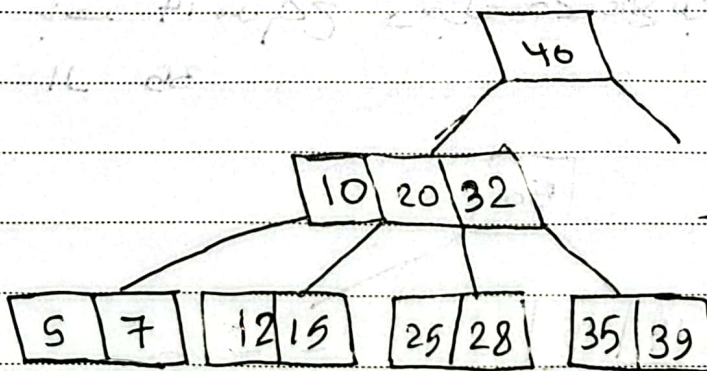
أمثلة:

مثال 1

$t = 2$

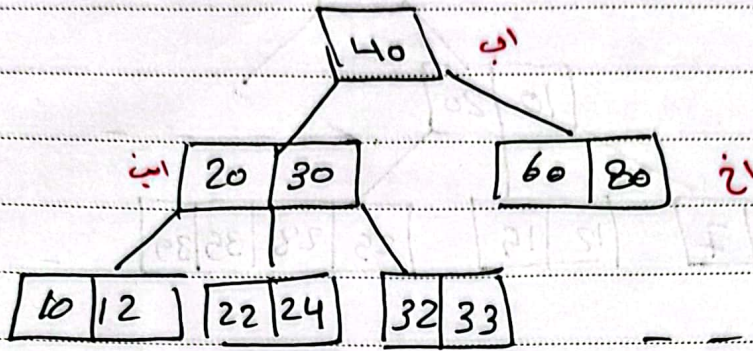


حذف 30 في صفحة عقدة داخلية ويوجد عنده تالي
 32 فين بدلوهو والصفحة فيها أكثر من t

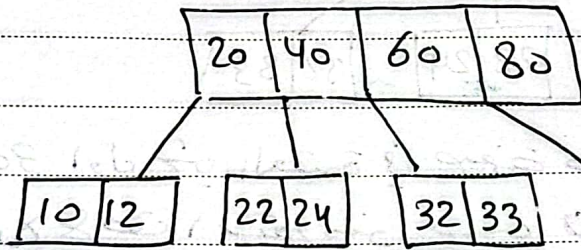


حذف 32 في صفحة عقدة داخلية ومافي عنده سابق
 أو لاحق فين بدلوهو مع الابداد حافظا على التوازن

مثال 3



حذف 30 حالة الصيغة عندها ابناء كالهذه عقدة
سبي بدمج الاخ والابن والاب وحذف



واخيراً النهائية